

Question Bank — Week 15

The Principles of Language Translation (Lecture 15) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the code	4
Part D — Trees and stacks: draw every step	6
Part E — Complexity analysis	7
Part F — Find and fix the bug	8
Part G — Write the code	9

Questions: [week15-questions.md](#). Commit to your own answer before reading one here. Every trace, tree and value below was produced by running the reference solution.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	b	M07	a	M13	c	M19	a
M02	c	M08	d	M14	b	M20	d
M03	a	M09	b	M15	a	M21	c
M04	d	M10	c	M16	d	M22	b
M05	c	M11	a	M17	b		
M06	b	M12	d	M18	c		

W15-M01 — b. Values are the evaluator's job (a), balanced parentheses and the tree the parser's (c, d). The lexer knows nothing about structure.

W15-M02 — c. ['12', '+', '3']. 12 is one token: the lexer keeps reading while the next character is a digit.

W15-M03 — a. Every token is valid, so the lexer accepts it (b); but after the +, `factor` finds *, which cannot start a factor: `ValueError: unexpected '*'`. Nothing reaches the evaluator (c).

W15-M04 — d. “Abstract” because it drops the grammar’s bookkeeping — the parentheses, and the calls that only passed a value up.

W15-M05 — c. An `expr` is a sequence of terms, and every * is consumed inside a term before `expr` sees the next +; so * ends up deeper in the tree. There is no table (a) — that was shunting-yard.

W15-M06 — b. $2 + (3 * 4) - 1 = 13$. 19 (a) ignores precedence: $((2 + 3) * 4) - 1$.

W15-M07 — a. $(2 * 3) + 4$: the last operator applied is the root.

W15-M08 — d. Each parenthesised group is a factor of the one term $(...) * (...)$. Parentheses (c) never appear in the tree.

W15-M09 — b. Left-associative: the leftmost operation first. $1 - (2 - 3)$ (c) would be 2.

W15-M10 — c. It builds $8 - (4 - 2) = 6$: the recursive call parses the **whole rest** of the line and hangs it on the right. The correct value is 2 (a).

W15-M11 — a. `expr` calls `expr` at the same position, before reading anything, so no call is ever closer to a base case. Python stops it at its recursion limit, so it does use memory — a frame per call (d is wrong).

W15-M12 — d. A while loop that runs while `peek()` is one of the operators. A rule name becomes a call (a); `|` becomes an `if` (c).

W15-M13 — c. One per number: 1, 2 and 3. (Measured: 3 calls.)

W15-M14 — b. Unary minus is a factor, so it binds only the 5: $(0 - 5) + 2 = -3$. Option (a) is the tree a parser builds if unary minus calls `expr` instead of `factor` (W15-B3). The lexer never produces negative numbers (c): -5 is two tokens.

W15-M15 — a. An operator can be applied only once both operand values are known: children first.

W15-M16 — d. Left subtree $3 - 4 +$, right subtree 2, then the root *. It is also the postfix form of Lecture 06.

W15-M17 — b. `expr` parses $3 + 4$ and stops at the `)`, which no rule can use; only the final check notices it. (a) and (c) fail inside `factor` anyway.

W15-M18 — c. $1,000 / 3$ is about 333, less a few frames already in use: measured from a script, 329 pairs work and 330 raise `RecursionError`.

W15-M19 — a. Each stage touches each token or node a constant number of times.

W15-M20 — d. The parser uses a loop (a is about the parser), but it builds a tree of height $n - 1$, and `evaluate` recurses once per level. $\log_2 n$ (b) is the balanced case.

W15-M21 — c. The bytecode is in `__pycache__`; `dis` shows it.

W15-M22 — b. Both routes are $O(n)$ (a). The tree route **needs** recursion (c), and memory for the tree (d).

Part B — Short answer and essay

W15-E1 (4)

- **Lexer:** text $\rightarrow$ tokens. Rejects characters outside the language: "3 \$ 4" $\rightarrow$ ValueError (unexpected character).
- **Parser:** tokens $\rightarrow$ tree (AST), following the grammar. Rejects valid tokens in an invalid order: "3 + * 4", "(3 + 4", "3 + 4)" $\rightarrow$ ValueError.
- **Evaluator:** tree $\rightarrow$ value. Rejects well-formed expressions with no value: "1 / 0" $\rightarrow$ ZeroDivisionError.
- **Compiler against interpreter:** a compiler translates the whole program ahead of time into another program (machine code, bytecode) that runs later; an interpreter executes as it goes and produces the result. Both share the lexer and parser; a compiler replaces the evaluator by a code generator. CPython does both: compile to bytecode, then interpret it.

W15-E2 (4)

- $\text{expr} := \text{term} (('+' | '-') \text{term})^*$ — an expression is a sum of terms; $\text{term} := \text{factor} (('*' | '/') \text{factor})^*$ — a term is a product of factors.
- To parse $3 + 4 * 2$, expr splits at $+$ into the terms 3 and $4 * 2$; the $*$ is consumed **inside** the second term. In the tree, $*$ is below $+$: $+$ over 3 and $*(4, 2)$. Deeper nodes are evaluated first, so $*$ “wins” — value 11.
- The lower the rule an operator lives in, the tighter it binds: precedence is **which rule** handles the operator, not a number.
- Parentheses: $\text{factor} := '(' \text{expr} ')'$ makes a whole parenthesised expression a **factor**, the tightest level, so $(3 + 4) * 2$ puts $+$ below $*$.
- A tighter operator such as $^$ gets a **new rule between term and factor**, so that term is built from it and it is built from factors (W15-C5).

W15-E3 (4)

- **Associativity** decides how operators of the **same** precedence group: left-associative $1 - 2 - 3 = (1 - 2) - 3 = -4$.
- The loop in expr does $\text{node} = \text{BinOp}(\text{op}, \text{node}, \text{self.term}())$: the tree built so far becomes the **left** child of each new operator, so the first operation ends up deepest and is evaluated first — left associativity.
- **Right recursion** ($\text{expr} := \text{term} ('+' \text{expr})?$) puts the recursive result — the whole rest of the line — on the right: $1 - 2 - 3$ becomes $1 - (2 - 3) = 2$. No error; just a wrong answer, and only for chains of $-$ or $/$ (W15-B1).
- **Left recursion** ($\text{expr} := \text{expr} '+' \text{term}$) calls expr first, with nothing consumed: the recursion makes no progress, and **every** input, even 3 , ends in RecursionError.
- The $(\dots)^*$ form with a loop gives left associativity **and** terminates.

W15-E4 (3)

- **Stack route:** `infix_to_postfix` (operator stack, precedence table) then `evaluate_postfix` (value stack). $O(n)$; no recursion, so no depth limit; builds nothing but a list.
- **Tree route:** Parser (recursive descent) then `evaluate` (post-order). $O(n)$ too; the tree can be reused — printed, simplified, compiled — and the grammar is easy to extend (unary minus is one case in factor; the Week 6 `infix_to_postfix` produces `5 -` for `-5`, which `evaluate_postfix` rejects). Its recursion depth follows the tree's height.
- **Connection:** the postfix list is exactly the **post-order** traversal of the tree. Shunting-yard emits the post-order of a tree it never builds.

W15-E5 (3)

- Each method call pushes a frame holding its own locals — including its node so far and where it is in its rule. At any moment the frames on the call stack are one path of the parse tree: “inside a factor, inside a term, inside an `expr...`”. Returning pops a frame and resumes the rule that called it. A hand-written parser would need its own explicit stack for this; recursion gets it for free.
- **Depth** is set by the **shape** of the input, not its length: three frames per level of parentheses (`expr`, `term`, `factor`); a flat chain `1 + 1 + ...` is a loop and adds nothing to the parser's depth (but see W15-K2 for `evaluate`).
- **Too deep:** Python's limit (1,000 frames by default) raises `RecursionError` — at about 330 nested pairs of parentheses. Real compilers raise the limit, or use an explicit stack for deep cases.

Part C — Trace the code

W15-T1. `['4.5', '*', '(', '10', '-', '3', ')']`. Start indices: `4.5` at 2, `*` at 5, `(` at 6, `10` at 7, `-` at 9, `3` at 10, `)` at 11. The two leading spaces (indices 0 and 1) and the trailing one (index 12) are skipped: they separate tokens but never become one. `4.5` is read as one token: from index 2, the scan continues over `.` and `5` and stops at `*`.

W15-T2.

```
expr    at 0 ('2')
  term  at 0 ('2')
    factor at 0 ('2')      -> Num(2)
    factor at 2 ('(')      (term consumed '*')
      expr at 3 ('3')
        term at 3 ('3')
          factor at 3 ('3') -> Num(3)
        term at 5 ('1')    (expr consumed '-')
          factor at 5 ('1') -> Num(1); ')' consumed by the outer factor
      term at 8 ('4')      (expr consumed '+')
```

factor at 8 ('4') -> Num(4)

Tree: `BinOp('+', BinOp('*', Num(2.0), BinOp('-', Num(3.0), Num(1.0))), Num(4.0))` — + at the root, * on its left with 2 and the - below it, 4 on its right. Value **8.0**. Eleven calls; the deepest point has 6 frames of the three methods.

W15-T3. The tree: * at the root; its left child /, whose children are -(8, 2) and +(1, 2); its right child 3.

Order	Node	Value
1	8	8
2	2	2
3	-	6
4	1	1
5	2	2
6	+	3
7	/	2
8	3	3
9	*	6

Result **6.0**. The labels in this order, 8 2 - 1 2 + / 3 *, are the postfix list.

W15-T4. `infix_to_postfix` gives ['2', '3', '1', '-', '*', '4', '+'].

Token	Value stack (top on the right)
2	2
3	2 3
1	2 3 1
-	2 2
*	4
4	4 4
+	8

The value is 8 (an int here; the tree route gives 8.0). The postfix list is the post-order of the tree of W15-T2: 2, then the subtree 3 1 -, then *, then 4, then the root +.

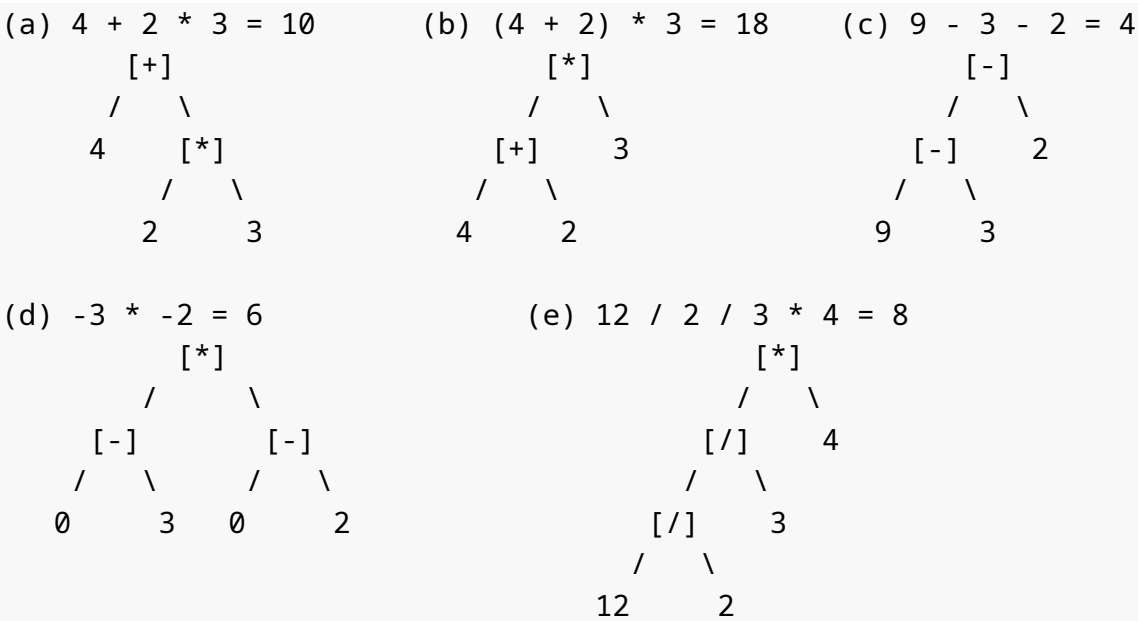
W15-T5.

Input	Result
"2 * -3"	-6.0 — unary minus is a factor, so it may follow *
"- -4"	4.0 — factor calls itself: 0 - (0 - 4)
"4 -"	ValueError from the parser : unexpected end of input

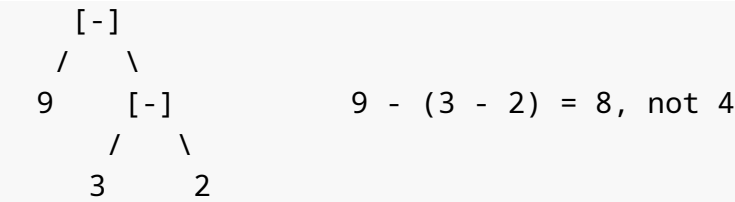
Input	Result
"(2 + 3) (4)"	ValueError from the parser : unexpected (after the expression — there is no implicit multiplication
"6 / (3 - 3)"	ZeroDivisionError from the evaluator
" "	ValueError from the parser : the lexer returns [], and parse rejects an empty expression
"2 ^ 3"	ValueError from the lexer : unexpected character ^ at position 2
"((2)"	ValueError from the parser : missing)

Part D — Trees and stacks: draw every step

W15-S1. Operators in brackets.

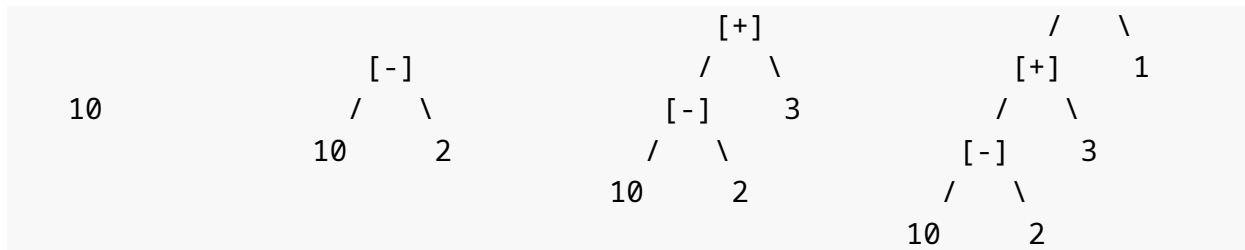


A right-recursive expr builds, for (c):



W15-S2. $10 - 2 + 3 - 1$:

after term():	after 1st turn:	after 2nd turn:	after 3rd turn:
			[-]



Each turn makes the old tree the **left** child of the new operator. Final value: $((10 - 2) + 3) - 1 = 10$.

W15-S3. Postfix: 1 2 + 3 4 - *.

Token	Value stack (top on the right)
1	1
2	1 2
+	3
3	3 3
4	3 3 4
-	3 -1
*	-3

The AST: * at the root, +(1, 2) on the left, -(3, 4) on the right. Post-order numbering: 1 → 1, 2 → 2, + → 3, 3 → 4, 4 → 5, - → 6, * → 7. **The post-order sequence is the postfix list**, and the values on the stack after each operator are the values evaluate returns at the same node: 3 at +, -1 at -, -3 at *. The stack holds exactly the values of the subtrees finished but not yet used.

Part E — Complexity analysis

W15-K1. Our tokenize: $\Theta(n)$. The index moves forward on every step of either loop and never back, so there are at most n steps of $O(1)$ work; each number is cut out once, with a slice as long as the number, so the slices total at most n characters. The `text = text[1:]` version: $\Theta(n^2)$. Each slice copies the rest of the string — $n - 1$, then $n - 2$, ... characters — and the sum is about $n^2/2$. Strings are immutable in Python, so “removing the first character” always means copying the others. (The same trap as slicing in recursion, Lecture 03 and W8-K2.)

W15-K2.

Input	(a) parser depth	(b) evaluate depth
k nested pairs around 1	about 3k — expr, term, factor per pair	1 — the tree is a single Num

Input	(a) parser depth	(b) evaluate depth
$1 + 1 + \dots + 1$, k numbers	constant — expr's loop	about k — the tree is left-deep, height $k - 1$
balanced, k numbers	about $3 \log_2 k$	about $\log_2 k$

For $k = 1,000$, two fail: the nested parentheses in the **parser** (3,000 frames; measured, it already fails at 330 pairs), and the flat chain in **evaluate** (about 1,000 frames; measured, the longest chain that works from a script has 997 numbers). The balanced one needs about 40 frames in all. Time is $O(n)$ in every case; depth is set by the **height** of the tree, as in Week 11.

W15-K3.

- **Concatenation:** each $+$ of lists builds a new list, copying both halves. A node whose subtrees hold L and R tokens costs $\Theta(L + R)$. For a **left-deep** tree (a chain like $1 + 1 + \dots + 1$), the left list grows by 2 at every level and is copied again at every level: $3 + 5 + 7 + \dots = \Theta(n^2)$. Measured: for 1,000 numbers (1,999 nodes) the concatenations copy 999,999 elements, to produce a list of 1,999.
- **One shared list:** each node appends one token, amortised $O(1)$ (Week 4): $\Theta(n)$.
- On a balanced tree the concatenating version is $\Theta(n \log n)$ — each token is copied once per level above it.

Part F — Find and fix the bug

W15-B1. Right recursion. The recursive call parses the whole rest of the line and hangs it on the **right**, so chains of $-$ group the wrong way: $10 - 4 - 3$ gives $10 - (4 - 3) = 9$ instead of 3, and $5 - 1 + 2$ gives $5 - (1 + 2) = 2$ instead of 6. No exception is raised, and chains of $+$ alone still give the right value ($1 + 2 + 3$ is 6 either way), which is why the bug survives casual testing. **Fix:** a loop that makes the tree so far the left child:

```
def expr(self):
    node = self.term()
    while self.peek() in ("+", "-"):
        op = self.advance()
        node = BinOp(op, node, self.term())
    return node
```

W15-B2. The $)$ is **checked but never consumed**. `factor` returns with position still on the $)$; the enclosing `term` and `expr` see a token they cannot use and return, and `parse` rejects it: every parenthesised expression — $(1 + 2) * 3$, $(5), 2 * (3)$ — raises `ValueError: unexpected ')' after the expression`. Expressions without parentheses still work. **Fix:** `self.advance()` after the check, before `return node`.

W15-B3. The operand of unary minus is parsed with `expr`, so the minus swallows the **whole rest** of the expression: $-2 + 5$ becomes $-(2 + 5) = -7$ instead of 3, and $2 * -3 + 1$

becomes $2 * -(3 + 1) = -8$ instead of -5 . It happens to be right for -5 and $-2 * 3$ (both -6 either way), which is why only the test for $-5 + 2$ catches it. **Fix:** `self.factor()` — unary minus binds as tightly as a number.

W15-B4. The inner loop reads `text[i]` without checking `i < len(text)` first. A number at the very **end** of the text runs the index off the end: `tokenize("3 + 4")` and `tokenize("12")` raise `IndexError: string index out of range`, while `"3 + 4 "` (trailing space) and `"(3)"` work — so the bug looks random. **Fix:** `while i < len(text) and (text[i] in DIGITS or text[i] == "."):` — the bound first, so and stops before the read. A second, quieter bug: nothing counts the decimal points, so `"1..2 "` becomes the token `'1..2'` and the error appears later, as a failed float conversion in the parser; count the points and raise in the lexer.

Part G — Write the code

W15-C1

```
def to_postfix(node):
    out = []

    def walk(current):
        if isinstance(current, Num):
            out.append(number_text(current.value))
        else:
            walk(current.left)      # left, right ...
            walk(current.right)
            out.append(current.op)   # ... then the node

    walk(node)
    return out
```

The walk visits the left subtree, then the right, then the node, appending as it goes: post-order. `infix_to_postfix` produces the same list because the shunting-yard algorithm outputs an operator exactly when both of its operands are complete — when its subtree is finished — which is when a post-order walk reaches it (`test_to_postfix_agrees_with_the_stack_route` checks five expressions). One shared list, so $\Theta(n)$; building the result by list concatenation would be $\Theta(n^2)$ on a left-deep tree (W15-K3).

W15-C2

```
def to_infix(node):
    if isinstance(node, Num):
        return number_text(node.value)
    return f"({to_infix(node.left)} {node.op} {to_infix(node.right)})"
```

One pair of parentheses per operator, never around a lone number. The output is unambiguous without any knowledge of precedence — which is exactly why it is ugly: $((1 - 2) - 3)$.

W15-C3

```
def to_infix_minimal(node):
    if isinstance(node, Num):
        return number_text(node.value)
    mine = PRECEDENCE[node.op]
    left = to_infix_minimal(node.left)
    right = to_infix_minimal(node.right)
    if isinstance(node.left, BinOp) and PRECEDENCE[node.left.op] < mine:
        left = f"({left})"
    if isinstance(node.right, BinOp) and PRECEDENCE[node.right.op] <= mine:
        right = f"({right})"          # equal too: trees lean left
    return f"{left} {node.op} {right}"
```

When a child needs parentheses: when its operator binds **more loosely** than its parent's (a + under a *), and — for the **right** child only — also when it binds **equally** (a - under a -). The grammar builds left-leaning trees, so an unparenthesised $1 - 2 - 3$ already means the left child; a right child of the same level can only be expressed with parentheses. That includes $1 + (2 + 3)$: mathematically the parentheses could go, but without them the text parses to a **different tree**, and the round-trip test (test_to_infix_minimal_round_trips, 300 random trees) demands the same one. A left child of equal precedence never needs them: $2 * 3 * (4 + 5)$.

W15-C4

```
def postfix_to_tree(tokens):
    trees = Stack()
    for token in tokens:
        if token in PRECEDENCE:
            if len(trees) < 2:
                raise ValueError(f"operator {token!r} needs two operands")
            right = trees.pop()          # popped first: the RIGHT one
            left = trees.pop()
            trees.push(BinOp(token, left, right))
        else:
            trees.push(Num(float(token)))
    if len(trees) != 1:
        raise ValueError("malformed postfix expression")
    return trees.pop()
```

evaluate_postfix with trees in place of numbers: a number becomes a leaf and is pushed; an operator pops **two** subtrees — the first popped is the **right** one — and pushes the tree that joins them. Exactly one tree must be left. Uses your Stack, as the storage rule asks. $\Theta(n)$, and postfix_to_tree(to_postfix(t)) == t for any tree (tested on

300 random ones).

W15-C5

```
class _PowerParser:

    def __init__(self, tokens):
        self.tokens = Array.from_values(tokens)
        self.position = 0

    def peek(self):
        if self.position < len(self.tokens):
            return self.tokens[self.position]
        return None

    def advance(self):
        token = self.peek()
        self.position += 1
        return token

    def expr(self):
        value = self.term()
        while self.peek() in ("+", "-"):
            if self.advance() == "+":
                value = value + self.term()
            else:
                value = value - self.term()
        return value

    def term(self):
        value = self.unary()
        while self.peek() in ("*", "/"):
            if self.advance() == "*":
                value = value * self.unary()
            else:
                value = value / self.unary()
        return value

    def unary(self):
        if self.peek() == "-":
            self.advance()
            return -self.unary()
        return self.power()

    def power(self):
        base = self.atom()
```

```

        if self.peek() == "^":
            self.advance()
            return base ** self.unary()           # recursion: leans RIGHT
        return base

    def atom(self):
        token = self.advance()
        if token is None:
            raise ValueError("unexpected end of input")
        if token == "(":
            value = self.expr()
            if self.advance() != ")":
                raise ValueError("missing ')')")
            return value
        if token[0] in "0123456789.":
            return float(token)
        raise ValueError(f"unexpected {token!r}")

def calculate_with_power(tokens):
    parser = _PowerParser(tokens)
    if parser.peek() is None:
        raise ValueError("empty expression")
    value = parser.expr()
    if parser.peek() is not None:
        left_over = parser.peek()
        raise ValueError(f"unexpected {left_over!r} after the expression")
    return value

```

Why recursion for ^: power parses one atom and then, if a ^ follows, calls unary for **everything to its right** — that call can itself reach power and take the next ^. So $2 \wedge 3 \wedge 2$ becomes $2 \wedge (3 \wedge 2)$: the recursive result goes on the right, and the tree leans **right**. This is the “right recursion” of W15-B1 — a bug for -, but exactly the right shape for a right-associative operator. + and * use loops because they lean left.

Precedence: unary sits **above** power, so in $-2 \wedge 2$ the minus applies to the whole power: $-(2^2) = -4$. And the right operand of ^ is a unary, so $2 \wedge -1$ is allowed and gives 0.5. Both match Python’s ** (test_calculate_with_power_matches_python). This version computes values as it parses rather than building a tree — a translator may do either — and keeps its tokens in the course Array.